

Scheduled Rules

A short primer: a calendar rule and a function, how one firing runs, what retries do to your node, and where an orchestrator begins

Written for the *score every account* lab, where the nightly run is one EventBridge rule with the scoring function as its target. The platform in the course is AWS; the mechanism is the same on every cloud and inside every orchestrator. Names and limits are October 2026 and will change; the mechanism will not.

1. What it is

EventBridge is an event bus: services put events on it, and **rules** match events and send them to **targets**. A **scheduled rule** is a rule whose event is the clock. You give it three things:

IN THE LAB	
a schedule expression : when to fire	<code>cron(0 3 * * ? *)</code> , every night at 03:00 UTC
a target : what to invoke	the scoring function
an input : the event the target receives	<code>{"batch": {"key": "accounts/nightly.jsonl"}}</code>

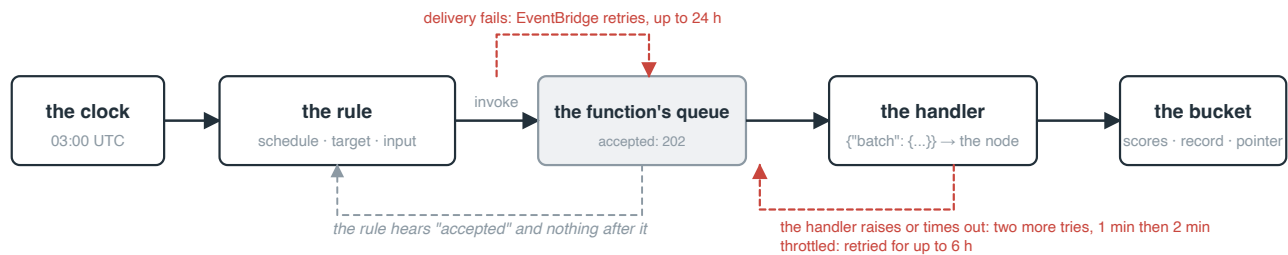
That is the whole object. There is no code in it, no knowledge of what the target does, and no memory of what happened last time. It is a calendar with one arrow on it.

2. How one firing runs

1. The clock reaches the scheduled minute. EventBridge makes an event. With a constant input, the event *is* your JSON; without one it is a small record saying which rule fired and when.
2. EventBridge invokes the target. For a function this is an **asynchronous** invocation: the event goes into the function's queue and EventBridge is told "accepted". It does not wait for the handler and never learns what the handler did.
3. The platform takes the event off the queue and runs the handler, cold or warm as usual. The handler sees the input, not an HTTP request: in the lab, a dictionary with a `batch` key, which it routes to the

node instead of to the web app.

4. The node does the work and writes its own record. In the lab that is the scores file, the run record, and the pointer, into the bucket. Nothing about the run is written by the rule.



so the node is written to be run twice

every row carries the run id; the scores file and the record for that id are replaced, never appended; the pointer moves only when the check passes. A retry, a duplicate delivery, or a hand re-run all land in the same place. When every retry is exhausted the event is dropped, and the morning reader finds the pointer still on yesterday.

3. The schedule expression

A **cron** expression has six fields: minutes, hours, day of month, month, day of week, year. Day of month and day of week cannot both be set; the one you are not using takes `?`. Times are UTC. A **rate** expression is an interval: `rate(1 minute)`, `rate(5 minutes)`, `rate(1 day)`. Nothing fires more often than once a minute.

EXPRESSION	FIRES
<code>cron(0 3 * * ? *)</code>	03:00 UTC every day: the lab's nightly run
<code>cron(0 3 ? * SUN *)</code>	03:00 UTC on Sundays: the lecture's weekly job
<code>cron(0/15 * * * ? *)</code>	every quarter hour
<code>rate(1 minute)</code>	once a minute from the moment the rule is created: for watching it fire

The rule fires at the minute, not the second; a few seconds of skew is normal. The **as-of** of the data the node reads is a different date from the one the rule fires on, and the run record carries both.

4. Retries, and why the node must be idempotent

Two layers retry on your behalf, and neither asks you first.

LAYER	WHEN	HOW LONG
EventBridge, delivering the event	the invoke call itself fails: the function is throttled, the service errors	up to 24 hours and 185 attempts, with exponential backoff
the function's queue, running the handler	the handler raises, or hits the time limit	two more attempts, one minute then two minutes later
the function's queue, when throttled	not enough concurrency to run it now	up to 6 hours, backing off to five minutes between tries

And the queue is only eventually consistent: a handler can receive the same event twice with no error anywhere. So a node that is run twice for the same night must produce the same thing it produced once. The lab's node keys everything by the run id and *replaces* the scores file and the record for that id, so a retry is harmless. A node that appended would score every account twice and a reader would never know which rows were real.

When the retries are exhausted the event is dropped, silently, unless a [dead-letter queue](#) is configured to catch it. The rule does not know; the function does not know; the morning's reader finds the pointer still on yesterday's run. That is why the lab asks what the page should do when last night's run did not publish.

5. Who may invoke the function

A rule cannot invoke a function until the function says it may. The lab's schedule script adds one statement to the function's [resource policy](#): the EventBridge service, acting for this rule's ARN, may call `Lambda:InvokeFunction`. Delete the rule and the statement is dead weight; create a second rule and it needs its own.

The newer [EventBridge Scheduler](#) works the other way round: the schedule carries a role it assumes, and that role must be allowed to invoke the function. Same effect, two directions, and the reason the lab uses the rule: the Learner Lab's fixed role is not written for the Scheduler to assume.

6. What it costs

A scheduled rule on the default bus costs nothing to fire; the Scheduler has a free allowance of fourteen million invocations a month and a dollar per million past it. What you pay for is the function's running time, exactly as for a request. The lab's night, two thousand accounts, is a few hundred milliseconds at 512 MB: a fraction of a cent, every night, forever. The cost of a nightly run is the work, never the calendar.

7. What a rule is not

A rule is not an orchestrator. Set the two side by side:

AN ORCHESTRATOR	A SCHEDULED RULE
starts the job on the schedule	starts one target on the schedule
runs each step when the steps it needs have finished	has no steps; up to five targets, fired side by side, in no order
retries a step that fails	the retries in section 4, which belong to delivery and to the function, not to the rule
stops everything downstream and tells someone	learns nothing after "accepted"; cannot stop what it does not know about
records every run, every step, start, end, outcome	records nothing; the node writes its own record

Everything in the right column is what the lab's node and bucket supply by hand: the run id, the check, the record, the pointer. An orchestrator supplies them for every step of every job, and that is the whole of what Airflow, Dagster, Prefect and Step Functions add. The mechanism underneath is still a clock, a target and an input.

8. Words

TERM	MEANING
event bus	where events are put and matched; the default bus is the one the clock writes to
rule	a match on the bus (here: a schedule) and the targets it sends to
target	what the rule invokes: a function, a queue, a state machine, up to five per rule
input	the event the target receives; a constant input replaces the clock's event with your JSON
asynchronous invocation	the event is queued and the caller is told "accepted"; the caller does not wait and does not learn the result
retry	another attempt, made for you, without asking: by EventBridge for delivery, by the function's queue for the handler
idempotent	run twice for the same id, the same result as once; the property retries require
dead-letter queue	where an event goes after every retry fails; without one it is dropped
resource policy	the function's own statement of who may invoke it; what the rule needs
as-of	the date the data is for, as distinct from the date the rule fired